Lecture 1: Performance Scaling and Amdahl's Law

1 Why parallel architectures?

1.1 Moore's law and Dennard scaling

Moore's law describes the historical growth in the number of transistors on integrated circuits, with a doubling roughly every two years. More transistors provide resources for additional cores, execution units, and caches; they do not automatically guarantee a proportional increase in performance.

Under classical Dennard scaling, shrinking transistors and scaling their voltages allowed faster switching without increasing power density. Computational throughput can be expressed as

$$\text{Performance [operations/s]} = \frac{\text{operations}}{\text{cycle}}\, f_{\text{clk}}. \tag{1}$$

Performance can improve through more useful operations per cycle or a higher clock frequency. Operations per cycle depend on the execution resources and their utilization. A deeper pipeline can help support higher frequency, but does not by itself multiply sustained operations per cycle.

1.2 The power wall

Increasing frequency is constrained by power consumption and heat dissipation. A simplified model separates dynamic switching power from static leakage power:

$$\boxed{P_{\text{power}} = \underbrace{\alpha C f_{\text{clk}} V^2}_{\text{dynamic power}} + \underbrace{V I_{\text{leakage}}}_{\text{static power}}.} \tag{2}$$

Here α is the switching activity factor, C the effective switched capacitance, f_{clk} the clock frequency, V the supply voltage, and I_{leakage} the leakage current.

At fixed voltage, dynamic power grows linearly with frequency. If higher frequencies require voltage to grow approximately proportionally to frequency, then, for fixed α and C,

$$P_{\text{dynamic}} \propto f_{\text{clk}} V^2, \quad V \propto f_{\text{clk}} \quad \Longrightarrow \quad P_{\text{dynamic}} \propto f_{\text{clk}}^3.$$

The cubic relation is an approximation under these assumptions, not a general law for total power. The resulting practical power and thermal constraint is called the **power wall**.

1.3 Post-Dennard design

When frequency scaling is limited, performance gains increasingly rely on parallelism and energy efficiency. Two important costs are the energy per operation and the energy required to move data between memory and processing units. This motivates using multiple cores or execution units, often at lower frequencies, instead of relying on one increasingly fast core. The benefit depends on how much useful work can actually run in parallel.

2 Measuring parallel performance

For a fixed problem size M, let $T_1(M)$ be the sequential execution time and $T_N(M)$ the execution time with N processing units. The speedup is

$$\boxed{S(N) = \frac{T_1(M)}{T_N(M)}.} \tag{3}$$

Speedup compares elapsed times; using more processors does not necessarily make a program faster.

Regime	Condition	Interpretation
Slowdown	$S(N) < 1$	Parallel execution takes longer
No improvement	$S(N) = 1$	Equal execution times
Sublinear speedup	$1 < S(N) < N$	Improvement below the processor count
Linear speedup	$S(N) = N$	Ideal proportional improvement
Superlinear speedup	$S(N) > N$	Additional benefits, such as better caching

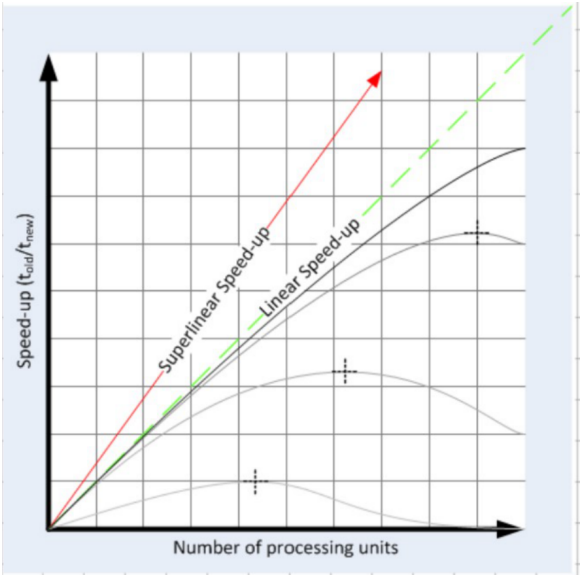


Figure 1: Speedup regimes as the number of processing units increases.

2.1 Parallel efficiency

Efficiency measures the speedup obtained per processing unit:

$$\boxed{E(N) = \frac{S(N)}{N} = \frac{T_1(M)}{N\,T_N(M)}.} \tag{4}$$

Linear speedup gives $E(N) = 1$; sublinear speedup gives $E(N) < 1$. Superlinear speedup gives $E(N) > 1$, which is possible when the parallel run benefits from architectural effects absent from the sequential baseline. Efficiency is a relative performance metric, not a direct measurement of processor utilization.

3 Amdahl's law

3.1 Assumptions and derivation

Amdahl's law describes the speedup of a fixed workload when only part of its original execution time can be accelerated. Let

- T_1 be the original sequential execution time;
- p be the parallelizable fraction of that time;
- $s = 1 - p$ be the serial fraction;
- N be the number of processing units.

The ideal parallel model assumes unchanged serial work, perfect load balancing, no parallel overhead, and an N-fold acceleration of the parallel portion. The execution time is the **sum** of the serial and parallel contributions:

$$T_{\text{serial}} = (1 - p)T_1, \tag{5}$$

$$T_{\text{parallel}} = \frac{pT_1}{N}, \tag{6}$$

$$T_N = T_1 \left((1 - p) + \frac{p}{N} \right). \tag{7}$$

Dividing the original time by the new time gives

$$\boxed{S(N) = \frac{T_1}{T_N} = \frac{1}{(1 - p) + p/N} = \frac{1}{s + p/N}.} \tag{8}$$

For normalized original time $T_1 = 1$, the new time is simply $(1 - p) + p/N$. The fractions refer to *execution time*, not to the fraction of source-code lines that can be parallelized.

3.2 Limits and diminishing returns

For $0 \leq p < 1$, the serial part imposes the limit

$$\boxed{\lim_{N \to \infty} S(N) = \frac{1}{1 - p} = \frac{1}{s}.} \tag{9}$$

For example, one hour of serial setup and nine hours of parallelizable work give $s = 0.1$, so the maximum ideal speedup is 10, regardless of how many processors are available.

For fixed $0 < p < 1$, adding processors increases ideal speedup with diminishing returns. If $p = 0$, there is no speedup; if $p = 1$, the ideal result is $S(N) = N$. Under this model, efficiency is

$$E(N) = \frac{1}{N(1 - p) + p}.$$

When a serial fraction remains, efficiency decreases as N grows.

3.3 General form: accelerating any portion

The same argument applies beyond parallel computing. If a fraction f of the original time is accelerated by a factor k, then

$$T_{\text{new}} = T_{\text{old}} \left((1 - f) + \frac{f}{k} \right), \qquad A(k) = \frac{1}{(1 - f) + f/k}.$$

The parallel form follows by setting $f = p$ and $k = N$.

4 Interpreting results in real applications

Amdahl's time decomposition is correct under its assumptions. The usual prediction with an N-fold acceleration of the parallel part is idealized: real execution can be either slower or faster than that prediction.

4.1 Why the prediction can be too optimistic

Parallel execution introduces communication, synchronization, thread creation, scheduling, and load-imbalance costs. A more realistic time model is

$$T_N = T_1 \left((1 - p) + \frac{p}{N} \right) + T_{\text{overhead}}(N). \tag{10}$$

With positive overhead, the actual speedup is below the ideal prediction. For example, a computation with an ideal 100× speedup can achieve less if processors spend substantial time exchanging intermediate results over a network. At sufficiently large processor counts, increasing communication costs can even make adding processors counterproductive.

4.2 Why the prediction can be too pessimistic

Partitioning data can change the memory behavior. A working set that does not fit in one processor's cache may fit into the combined caches of several processors. Fewer slow memory accesses can accelerate the parallel portion by more than N, producing a larger speedup than Equation (8) predicts and potentially even superlinear speedup.

This does not invalidate the decomposition of execution time. It means that the assumed acceleration of the improved portion must reflect the actual hardware behavior rather than simply being set equal to N.

4.3 Fixed versus growing workloads

Strong scaling asks how the execution time changes when the processor count increases while the total problem size stays fixed. This is the setting of the parallel Amdahl model.

Weak scaling increases total work with processor count, keeping the work per processor approximately fixed, and examines how execution time changes. For example, a weather simulation can use more processors to increase grid resolution rather than to finish the same simulation sooner.

A related fixed-time question asks how much larger a problem can be solved in the same elapsed time. Gustafson's law describes scaled speedup using the serial fraction measured in a parallel run. Its serial fraction need not equal the fraction measured in the original fixed-size sequential run. These workload comparisons do not contradict Amdahl's fixed-workload limit.

4.4 Practical interpretation

Before comparing speedups, check that the workload and sequential baseline are comparable, identify communication and synchronization costs, and consider whether cache behavior changes. A processor count alone is insufficient to predict application performance.

5 Worked examples and revision

5.1 Case A: $p = 0.5$, $N = 4$

$$\frac{T_4}{T_1} = 0.5 + \frac{0.5}{4} = 0.625, \qquad S_A = \frac{1}{0.625} = \boxed{1.6}.$$

Efficiency is $E_A = 1.6/4 = 0.4 = 40\%$, and the asymptotic speedup limit is 2.

5.2 Case B: $p = 0.2$, $N = 8$

$$\frac{T_8}{T_1} = 0.8 + \frac{0.2}{8} = 0.825, \qquad S_B = \frac{1}{0.825} = \boxed{\frac{40}{33} \approx 1.212}.$$

Efficiency is $E_B = S_B/8 = 5/33 \approx 15.15\%$, and the asymptotic limit is 1.25.

5.3 What the comparison shows

Case A has a larger speedup despite using fewer processors, because a larger fraction of its original time can be parallelized. Case B is dominated by its 80% serial fraction. Since both p and N change, these cases do not isolate the effect of processor count.

Keeping $p = 0.5$ and changing only N from four to eight instead gives

$$S(8) = \frac{1}{0.5 + 0.5/8} = \frac{16}{9} \approx 1.778 > 1.6.$$

More processors help for this fixed p, but the improvement is limited by the same serial bottleneck.

5.4 Key formulas

Quantity	Formula
Measured speedup	$S(N) = T_1/T_N$
Parallel efficiency	$E(N) = S(N)/N$
Ideal parallel time	$T_N = T_1\big((1-p) + p/N\big)$
Amdahl speedup	$S(N) = 1/\big((1-p) + p/N\big)$
Serial-fraction limit	$S_{\max} = 1/(1-p)$, for $p < 1$

5.5 Checks to remember

- Add the serial and parallel execution times; do not multiply them.
- Distinguish the speedup of one portion from the overall speedup.
- Check whether the problem size is fixed before applying an Amdahl limit.
- State the voltage assumption before using a cubic frequency–power relation.
- Explain superlinear speedup through changes in execution behavior, such as cache effects, rather than treating it as impossible.